

성능개선

지금까지는 모델을 돌려서 점수를 봤습니다.
이제 그 점수를 더 믿을 수 있게, 더 높게 만듭니다.

Q1 이 단원이 다루는 것

앞 단원들이 "일단 돌아가게 만드는 법" 이었다면, 여기는 "제대로 만드는 법" 입니다. 새 모델을 배우는 게 아니라, 이미 만든 모델에 얹는 도구 네 개를 배웁니다.

교차검증

점수를
믿을 수 있게

정규화

통째로 외우기
막기

튜닝

최적 설정
자동으로 찾기

파이프라인

시험지 훑쳐보기
차단

네 개 모두 '학습·검증' 단계에 얹는 도구입니다

예제는 당뇨 환자 442명의 신체 지표 10개(나이·체질량지수·혈압·혈액검사 6종)로 1년 뒤 병 진행도를 예측하는 문제입니다. 답이 연속된 숫자니까 회귀고, 채점은 R^2 로 합니다.

Q2 먼저 몸무게부터 재기 — baseline

baseline = 다이어트 시작 전에 재 두는 몸무게

이게 있어야 나중에 "몇 kg 뺐다"를 말할 수 있습니다

기법을 쓰기 전의 점수를 먼저 기록해둡니다. 안 그러면 나중에 뭐가 얼마나 좋아졌는지 알 수가 없어요.

```
X_tr, X_val, y_tr, y_val = train_test_split(X, y, test_size=0.2, random_state=0)
lin = LinearRegression().fit(X_tr, y_tr)
print('단일 분할 R2:', r2_score(y_val, lin.predict(X_val))) # 0.3322
```

그런데 이 0.33이라는 점수, 사실 못 믿습니다. 왜 그런지가 다음 이야기예요.

Q3 교차검증 — 시험을 한 번만 보면 안 되는 이유

데이터를 한 번 잘라서 80%로 공부하고 20%로 시험을 봤습니다. 그런데 하필 그 20%에 쉬운 문제만 몰렸다면? 반대로 어려운 문제만 몰렸다면?

시험을 딱 한 번만 보면 **그날의 운에** 점수가 휘둘립니다

그래서 **모의고사를 5번 보고 평균**을 냅니다. 이게 교차검증이에요.

- 데이터를 5조각으로 나눈다
- 1회차 — **1번 조각**을 시험지로, 나머지 4조각으로 공부
- 2회차 — **2번 조각**을 시험지로, 나머지 4조각으로 공부
- ... 모든 조각이 딱 한 번씩 시험지가 되도록 5번 반복

점수가 5개 나오면 그 평균을 봅니다. 좋은 회차와 나쁜 회차의 운이 **평균에서 서로 상쇄**해서 진짜 실력에 가까워져요.

조각을 몇 개로 나눌까

빨간 칸이 그 회차의 시험지, 회색이 공부용입니다. 조각 수를 바꿔보세요.

1회차					0.55
2회차					0.43
3회차					0.51
4회차					0.47
5회차					0.53

조각 수 (cv)

5조각

회차별 점수
0.55 0.43 0.51 0.47 0.53

평균 R^2
0.4973

학습 횟수
5번

가장 흔한 설정입니다. 회차별로 0.43~0.55까지 출렁이지만 평균은 안정적이예요.

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(LinearRegression(), X, y, cv=5, scoring='r2')
print('5개 점수:', scores.round(3)) # [0.43 ... 0.55] 사이에서 출력
print('평균 R2:', scores.mean()) # 0.4823
```

한 번 낸 점수는 **0.33**, 5번 평균은 **0.48**.

모델은 그대로인데 점수가 이렇게 다릅니다. **0.33**은 운이 나빴던 한 번의 값이고, **0.48**이 이 모델의 진짜 실력이에요.

시간은 5배 들지만(5번 학습), "이 점수 믿어도 되나?"라는 불안이 사라집니다.

조각 수는 보통 **5나 10**을 씁니다. 데이터가 적으면 10으로 촘촘히, 많으면 5로 시간을 아껴요. 442개 데이터에 100조각처럼 너무 잘게 나누면 조각 하나가 너무 작아져서 점수가 오히려 들쭉날쭉해집니다.

Q4 과적합 — 답만 통째로 외운 학생

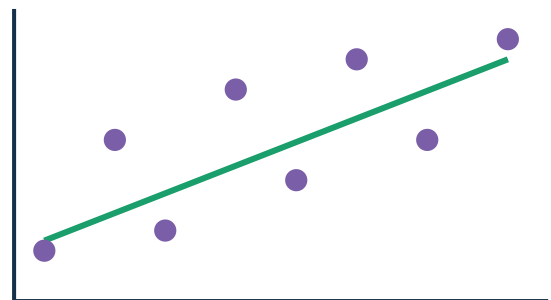
시험공부를 하면서 문제를 **이해하지 않고 답만 외운** 학생을 떠올려봅시다. 문제집은 100점인데 처음 보는 시험은 망하죠.

통째로 외운 모델 (과적합)



점을 하나도 안 놓치려고 구불구불
훈련 100점 · 실전 낙제

이해한 모델



몇 개는 틀리지만 흐름을 잡음
처음 보는 문제도 잘 풀

왼쪽은 훈련 데이터의 **사소한 우연과 잡음까지 외워버린** 상태입니다.

모델은 각 특성(체질량지수, 나이, 혈압...)에 **영향력 점수 = 계수**를 매깁니다. 과적합한 모델은 잡음까지 억지로 맞히려고 **이 계수를 지나치게 크게 키웁니다**.

Q5 정규화 — 계수에 벌점 매기기

정규화 = "계수를 너무 크게 매기면 벌점!"

억지로 눌러서 모델을 단순하게 만드는 규칙

벌점의 세기를 `alpha` 라고 부릅니다. 클수록 세게 누르죠. 누르는 방식이 두 가지 있습니다.

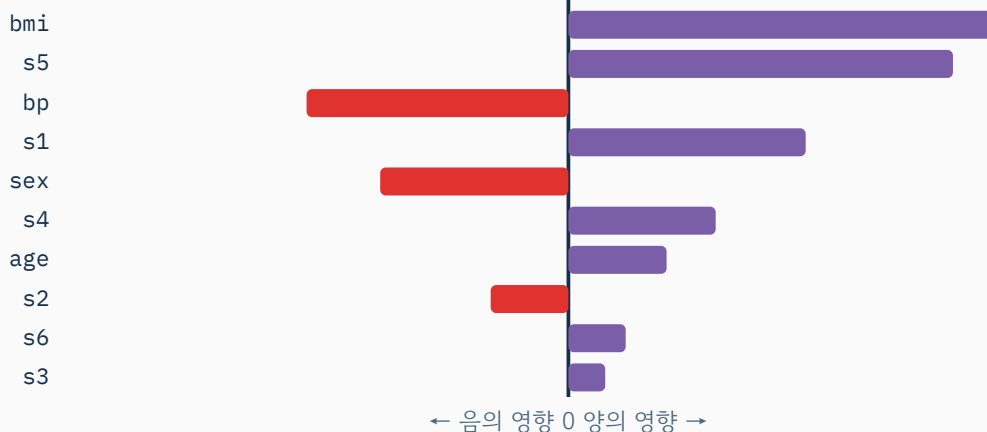
이름	하는 일	비유
Ridge	모든 계수를 조금씩 낮춤	다 살리되 안전하게
Lasso	필요 없는 특성 계수를 아예 0으로	그 특성을 자동으로 버림

벌점 세기(alpha) 조절해보기

막대는 특성 10개의 영향력(계수)입니다. 벌점을 올리면 막대가 눌립니다.

Ridge

Lasso



벌점 세기 `alpha`

0.0001

예상 R^2 점수

0.442

과적합 위험

살아남은 특성

10 / 10

벌점이 거의 없습니다. 계수가 커서 잡음까지 외울 수 있어요.

벌점은 셀수록 좋은 게 아닙니다. 너무 세면 모델이 아무것도 못 배워서 점수가 오히려 떨어져요(과소적합).

그래서 딱 맞는 **alpha**를 찾아야 하는데, 그게 다음 이야기입니다.

```
from sklearn.linear_model import Ridge, Lasso

for name, model in [('Ridge', Ridge(alpha=1)), ('Lasso', Lasso(alpha=0.1))]:
    s = cross_val_score(model, X, y, cv=5, scoring='r2').mean()
    print(name, s)
```

Q6 튜닝 — 오븐 다이얼 자동으로 맞추기

하이퍼파라미터는 모델이 스스로 못 정하는, 내가 미리 정해줘야 하는 설정값입니다. 요리로 치면 오븐의 온도와 시간 같은 것. 재료가 아니라 내가 맞추는 다이얼이예요. (여기선 alpha가 그 다이얼)

문제는 alpha에 0.01을 넣어야 할지 1을 넣어야 할지 100을 넣어야 할지 **미리 알 수 없다**는 것. 손으로 하나씩 넣고 점수 적고 비교하는 건 너무 번거롭죠.

후보 5개 X 교차검증 5조각 = 25번 학습						
	1회	2회	3회	4회	5회	평균
alpha 0.01	✓	✓	✓	✓	✓	0.527 ← 우승
alpha 0.1	✓	✓	✓	✓	.	0.518
alpha 1	✓	✓	.	✓	.	0.483
alpha 10	✓	.	.	.	.	0.401
alpha 100	.	.	.	.	.	0.152

각 칸 하나가 '학습 1번'. 이걸 다 돌려보고 평균이 가장 높은 줄을 고릅니다.

```
from sklearn.model_selection import GridSearchCV

grid = GridSearchCV(Ridge(), {'alpha': [0.01, 0.1, 1, 10, 100]}, cv=5, scoring='r2')
grid.fit(X_tr, y_tr)
print('최적 alpha:', grid.best_params_) # {'alpha': 0.01}
print('최적 CV R2:', grid.best_score_)  # 0.5267
```

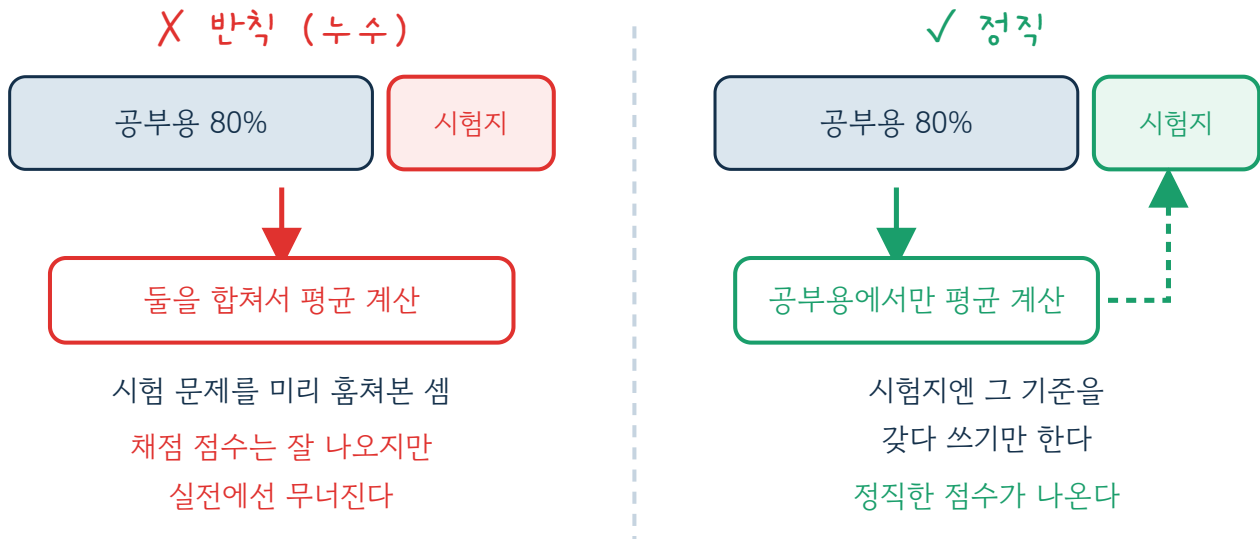
후보를 어떻게 줄까?

alpha 같은 값은 **10배씩 건너뛰며** 넓게 줍니다 — 0.001, 0.01, 0.1, 1, 10, 100
0.01, 0.02, 0.03처럼 촘촘히 주는 건 비효율이에요. **자릿수가 중요하지 소수점은 별로 안 중요**하거든요.

최적값이 후보의 **끝에 걸리면**(예: 제일 작은 0.01이 뺄이면) 그 방향으로 더 작은 값을 추가해서 다시 돌려봐야 합니다.

Q7 파이프라인 — 시험지 훑쳐보기 막기

스케일링은 평균과 표준편차를 구해서 값 크기를 맞추는 작업입니다. 그런데 이 평균을 전체 데이터로 구하면 어떻게 될까요?



시험 답안지를 미리 본 상태로 공부하는 것 — 이걸 **데이터 누수**라고 합니다.

문제는 **교차검증은 5번을 돈다**는 것. 매 회차마다 손으로 이걸 챙기면 언젠가 반드시 실수합니다.

파이프라인 = 전처리와 모델을 한 상자에 담기

상자가 알아서 매 회차 공부용에만 기준을 구해줍니다

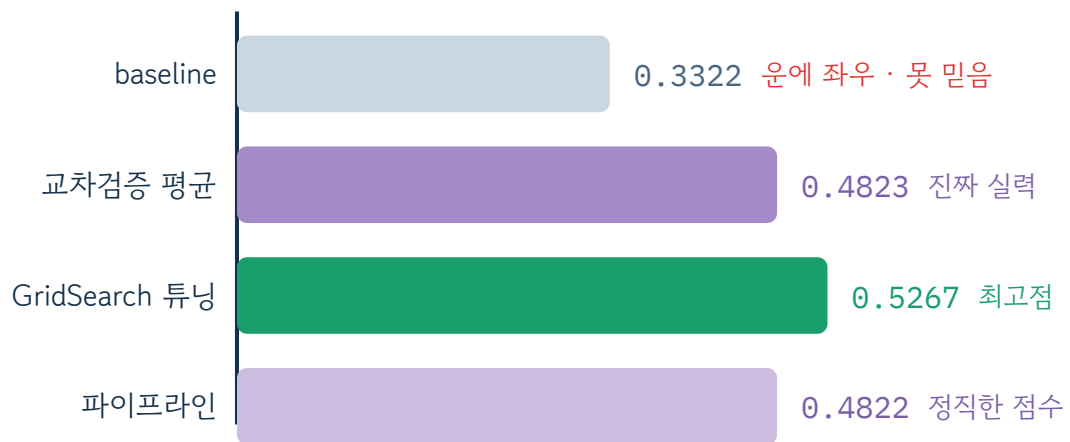
```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

pipe = make_pipeline(StandardScaler(), Ridge(alpha=1)) # 상자에 담기
cross_val_score(pipe, X, y, cv=5, scoring='r2').mean() # 0.4822
```

파이프라인 점수는 **0.4822**. 안 쓴 것(0.4823)과 거의 같습니다.

여기서 볼 건 점수가 아니라 "누수 없이 얻은 정직한 점수"라는 사실이에요. 반칙으로 얻은 높은 점수는 실전에서 무너지니까요.

Q8 결과 비교



모델은 그대로. 학습·검증 방법만 바뀌서 얻은 차이입니다.

0.33 → 0.48(믿을 수 있게) → 0.53(더 높게). 새 모델을 가져온 게 아니라 기존 모델에 도구 네 개를 엮었을 뿐입니다.

Q9 한 장 정리

기 법	한 줄 로	코 드
baseline	다이어트 전 몸무게	<code>train_test_split</code>
교차검증	모의고사 5번 평균	<code>cross_val_score(cv=5)</code>
정규화	통째로 외우면 벌점	<code>Ridge / Lasso(alpha)</code>
튜닝	오븐 다이얼 자동 탐색	<code>GridSearchCV</code>
파이프라인	전처리+모델을 한 상자에	<code>make_pipeline</code>

이것만 기억하면 됩니다

- ① 한 번 낸 점수는 **윤**이다 — 여러 번 재서 평균을 보라
- ② 점수가 안 오르는 것보다 **반칙으로 오른 점수**가 더 위험하다
- ③ 설정값은 손으로 찍지 말고 **후보를 주고 컴퓨터가 고르게** 해라

연습해볼 것

- ① Lasso로 GridSearchCV를 돌려 최적 alpha를 찾고 Ridge와 비교해보기
- ② `cv=5` 를 `cv=10` 으로 바꾸면 평균 R^2 가 어떻게 달라지나
- ③ GridSearchCV에 설정값을 두 개 이상 동시에 넣어 탐색해보기

Q10 Level별 코드 작성하기

옆에 주피터 노트북을 열고, 복사 버튼으로 코드를 붙여넣어 따라 해보세요. 쉬운 것부터 실전(캐글 공공데이터)까지 3단계입니다.

코드 해석 ▼을 누르면 한 줄씩 쉽게 풀어 줍니다.

LEVEL 1 단순 — baseline(한 번 나눈 점수)

```
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

X, y = load_diabetes(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=0)
base = r2_score(y_te, LinearRegression().fit(X_tr, y_tr).predict(X_te))
print('baseline R2 (한 번 나눔):', round(base, 3))
```

train_test_split -> r2_score

딱 한 번 나눠 R2를 잡니다. 이게 출발점(baseline).

한 번 값의 약점

이 점수는 '어떻게 나눴느냐'에 흔들립니다 -> 다음은 교차검증.

LEVEL 2 복잡 — 교차검증 + GridSearchCV 튜닝

```
from sklearn.datasets import load_diabetes
from sklearn.model_selection import cross_val_score, GridSearchCV
from sklearn.linear_model import Ridge

X, y = load_diabetes(return_X_y=True)
cv = cross_val_score(Ridge(alpha=1), X, y, cv=5, scoring='r2')
print('교차검증 평균 R2:', round(cv.mean(), 3))

grid = GridSearchCV(Ridge(), {'alpha': [0.01, 0.1, 1, 10, 100]}, cv=5, scoring='r2')
grid.fit(X, y)
print('최적 alpha:', grid.best_params_, '| 최적 R2:', round(grid.best_score_, 3))
```

```
cross_val_score(..., cv=5)
```

5조각으로 나눠 5번 채점한 평균 -> 믿을 수 있는 점수.

```
GridSearchCV(Ridge(), {'alpha': [...]} )
```

alpha 후보들을 자동으로 다 시도해 최고를 고릅니다.

```
grid.best_params_ / best_score_
```

최적 설정과 그때의 점수.

```
import pandas as pd
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge
from sklearn.model_selection import GridSearchCV

url = 'https://raw.githubusercontent.com/stedy/Machine-Learning-with-R-datasets/master/insurance.csv'
df = pd.read_csv(url)
X = pd.get_dummies(df.drop(columns='charges'))
y = df['charges']

pipe = make_pipeline(StandardScaler(), Ridge()) # 전처리+모델 한 상자(누수 방지)
grid = GridSearchCV(pipe, {'ridge__alpha': [0.1, 1, 10, 100]}, cv=5, scoring='r2')
grid.fit(X, y)
print('최적:', grid.best_params_, ' | 교차검증 R2:', round(grid.best_score_, 3))
```

```
make_pipeline(StandardScaler(), Ridge())
```

전처리와 모델을 한 상자로 -> 교차검증 중 누수 자동 차단.

```
GridSearchCV(pipe, {'ridge__alpha': [...]}))
```

파이프라인 통째로 튜닝('스텝이름_파라미터' 형식).

```
grid.best_params_
```

최적 alpha와 교차검증 R2를 봅니다.